# Práctica 1:

Heurística y optimización

Práctica: *Programación Lineal.*
**Heurística y Optimización**
Curso 2025-2026
Grupo 80

**Alumno 1**: Diego Valladares (100475849)
**Alumno 2**: María Robledano (100474933)

# Índice:

**1. Introducción** **3**
**2. Descripción de Modelos** **3**
2.1. Modelo Básico en Calc 3
Planteamiento del problema 3
Variables de decisión 3
Función objetivo 4
Restricciones 4
Implementación en Calc 4
2.2 Parte 2: Modelo avanzado en GLPK 6
Minimización del impacto de averías 6
Planteamiento del problema 6
Variables de decisión 7
Parámetros 7
Función objetivo 7
Restricciones 7
Implementación en GLPK 8
Maximización de la satisfacción de los pasajeros 9
Planteamiento del problema 9
Variables de decisión 9
Parámetros 9
Función objetivo 10
Restricciones 10
2.3. Parte 3: Análisis de Resultados 12

# 1. Introducción

En este documento hemos desarrollado la modelización y resolución de un problema de asignación y optimización de recursos en una compañía nacional de autobuses. El objetivo principal de la práctica es aplicar los conceptos de Programación Lineal y Entera para optimizar las decisiones operativas de la empresa, utilizando dos entornos de trabajo complementarios: LibreOffice Calc y GLPK (GNU Linear Programming Kit).

La práctica se divide en tres partes. En primer lugar, hemos planteado un modelo básico en Calc, orientado a resolver un problema de asignación mínima entre autobuses y talleres, con el propósito de reducir el impacto de las averías minimizando las distancias totales recorridas.
Posteriormente, hemos desarrollado dos modelos avanzados en GLPK, que amplían el problema a escenarios más generales, incorporando parámetros adicionales como el número de pasajeros, penalizaciones por inasistencia y franjas horarias disponibles.
Por último, hemos realizado un análisis de resultados, donde evaluamos las soluciones obtenidas, las restricciones activas y la complejidad de los modelos implementados.

# 2. Descripción de Modelos

Comenzamos con el modelo básico en Calc, centrado en un problema de asignación elemental, y posteriormente avanzamos hacia los modelos en GLPK, de carácter más general y automatizable.
Cada modelo se presenta definiendo las variables, parámetros, función objetivo, restricciones y metodología empleada para su resolución, con el fin de ofrecer una visión clara y estructurada del proceso de optimización realizado.

## 2.1. Modelo Básico en Calc

### *Planteamiento del problema*

En esta primera parte modelizamos un problema clásico de asignación: tenemos cinco autobuses ($a_1$, $a_2$, $a_3$, $a_4$, $a_5$) y cinco talleres ($t_1$,$t_2$,$t_3$,$t_4$,$t_5$) con distancias $c_{ij}$ conocidas. Buscamos minimizar la distancia total asignando cada autobús a exactamente un taller y garantizando que cada taller reciba solo un autobús. Es, por tanto, un caso estándar de programación lineal de coste mínimo.

### *Variables de decisión*

Hemos definido la variable de decisión $x_{ij}$ de la siguiente manera:

$$X_{ij} = \begin{cases} 1 & \text{si el autobús } a_i \text{ se asigna al taller } t_j \\ 0 & \text{en caso contrario} \end{cases} \qquad \text{donde } i, j = 1,2,3,4,5$$

Estas variables son binarias, porque un autobús sólo puede asignarse o no a un determinado taller.

### Parámetros

*$c_{ij}$: distancia (en km) entre el autobús $a_i$ y el taller $t_j$*
*i: índice de los autobuses (1 – 5).*
*j: índice de los talleres (1 – 5).*

### *Función objetivo*

El modelo tiene como objetivo minimizar la distancia total recorrida por todos los autobuses:

$$\boxed{min\ Z = \sum_{i=1}^{5} \sum_{i=j}^{5} c_{ij} x_{ij}}$$

El valor de Z representa la distancia total recorrida por todos los autobuses hasta sus respectivos talleres. Minimizar Z permite reducir el impacto de las averías y los costes asociados a los desplazamientos.

### *Restricciones*

1. **Asignación única de autobuses:**

$$\boxed{\sum_{j=1}^{5} x_{ij} = 1 \quad \forall i}$$ Cada autobús debe ser asignado exactamente a un taller.

2. **Capacidad única de los talleres:**

$$\boxed{\sum_{i=1}^{5} x_{ij} = 1 \quad \forall j}$$ Cada taller sólo puede atender a un autobús.

3. **Restricción de dominio:**

$$\boxed{x_{ij} \in \{0,\ 1\}}$$ Las variables son binarias, ya que no existen asignaciones parciales.

### *Implementación en Calc*

Para resolver el modelo de asignación que planteamos en la primera parte, utilizamos LibreOffice Calc, aunque en nuestro caso realizamos la implementación inicial en Microsoft Excel, ya que ambas herramientas funcionan de manera muy similar y permiten configurar un solver lineal para este tipo de problemas.

A continuación, explicamos los pasos que seguimos para construir el modelo, introducir las fórmulas y configurar el **solver** hasta obtener la solución óptima.

1. <u>Organización de los datos</u>

   En primer lugar, hemos añadido la tabla 1 del documento donde aparecen las distancias $c_{ij}$ entre cada autobús y taller. Esta tabla la situamos en el rango B4:G9. De esta forma, dejamos clara la correspondencia entre cada autobús y taller, y tenemos todos los datos listos para calcular el coste total de las posibles asignaciones.

2. <u>Definición de las variables de decisión</u>

   Justo debajo de la tabla anterior, creamos una segunda tabla del mismo tamaño que la anterior,

en el rango C14:G18.

Cada una de estas celdas representa una **variable binaria $x_{ij}$**, que indica si el autobús $a_i$ se asigna o no al taller $t_j$. Inicialmente, dejamos todas las celdas con valor 0.

De esta manera, el solver puede decidir qué valores deben cambiar a 1 para lograr la asignación óptima.

3. <u>Cálculo de sumas por fila y por columna</u>

   A continuación, añadimos las fórmulas necesarias para las restricciones del modelo:

   - En la columna H, colocamos la suma de cada fila (una por cada taller) usando la fórmula: **=SUM(C14:G14)**

     y la copiamos hacia abajo hasta H18.

     Estas celdas nos indican cuántos autobuses están asignados a cada taller.
   - En la fila 19, añadimos las sumas por columna (una por cada autobús) con la fórmula: **=SUM(C14:C18)**

     y la copiamos hacia la derecha hasta la columna G.

     Estas celdas muestran si un autobús ha sido asignado a algún taller.

4. <u>Función objetivo</u>

   Una vez definidas las variables, calculamos la función objetivo, que representa la distancia total recorrida.

   Para ello, utilizamos la función SUMPRODUCT, que multiplica cada celda de la matriz de distancias por su correspondiente variable de decisión, y luego suma todos los resultados.

   La fórmula que escribimos fue: **=SUMPRODUCT(B2:F6;B10:F14)**

   El resultado de esta fórmula lo colocamos en una celda aparte, identificada como "Función objetivo", para poder usarla fácilmente en el solver.

5. <u>Configuración del solver</u>

   Con todos los elementos definidos, abrimos la herramienta Solver desde el menú.

   Configuramos los siguientes parámetros:

   - Set objective: **$B$22**, la celda donde se calcula la función objetivo. En este caso como se muestra en el archivo que hemos incluido es donde aparece la función SUMPRODUCT.
   - To: seleccionamos la opción Min.
   - By Changing Variable Cells: el rango que contiene las variables binarias (**$C$14:$G$18**)
   - Restricciones:

     Cada taller sólo puede atender a un autobús → $H$14:$H$18 = 1

     Cada autobús debe asignarse a un único taller → $C$19:$G$19 = 1

     Todas las variables son binarias → $C$14:$G$18  bin

- Finalmente, escogimos el método de resolución Simplex LP, adecuado para problemas lineales enteros, y ejecutamos el solver.

6. Resultados obtenidos

Tras ejecutar el solver, obtuvimos la siguiente asignación óptima:

| | a1 | a2 | a3 | a4 | a5 |
|---|---|---|---|---|---|
| t1 | 0 | 1 | 0 | 0 | 0 |
| t2 | 0 | 0 | 0 | 0 | 1 |
| t3 | 0 | 0 | 1 | 0 | 0 |
| t4 | 0 | 0 | 0 | 1 | 0 |
| t5 | 1 | 0 | 0 | 0 | 0 |

Obtuvimos $Z = 573$ km como distancia mínima total. La solución cumple todas las restricciones: cada autobús está asignado a un único taller, cada taller recibe exactamente un autobús y $x_{ij} \in \{0,1\}$. Por tanto, podemos afirmar que la solución es **factible y óptima**.

Además, decidimos emplear la función **SUMPRODUCT** porque permite representar de forma directa y compacta la función objetivo del modelo de asignación. Esta herramienta facilita el cálculo del coste total al multiplicar cada elemento de la matriz de distancias por su variable binaria correspondiente, evitando tener que escribir fórmulas individuales para cada celda. De este modo, conseguimos una expresión algebraica equivalente a la formulación teórica $\sum_{i=1}^{5}\sum_{j=1}^{5} c_{ij}x_{ij}$ totalmente automatizada dentro de la hoja de cálculo.

Para verificar manualmente la corrección de la solución obtenida, revisamos que las celdas con valor 1 en la tabla de asignaciones coincidieran con las combinaciones que generaban las distancias mínimas posibles.

## 2.2 Parte 2: Modelo avanzado en GLPK

### Minimización del impacto de averías

#### *Planteamiento del problema*

En esta segunda parte de la práctica hemos desarrollado un modelo de Programación Lineal Entera que permite minimizar el impacto económico de las averías que se producen al inicio del servicio.

A diferencia del modelo básico, en este caso consideramos un único taller que dispone de varias franjas horarias de atención, y un conjunto de autobuses que pueden requerir asistencia. Cada autobús puede ocupar como máximo una franja horaria, y cada franja sólo puede asignarse a un autobús.
Además, se introduce un coste de desplazamiento proporcional a la distancia entre el autobús y el taller, y una penalización asociada a los pasajeros afectados en caso de que un autobús no pueda ser atendido. El objetivo final del modelo es minimizar el coste total, combinando ambos factores: los kilómetros recorridos por los autobuses que sí reciben asistencia y las penalizaciones de aquellos que no son atendidos.

### *Variables de decisión*

Hemos definido las siguientes variables:

$$X_{ij} = \begin{cases} 1 & \text{si el autobús } a_i \text{ se asigna en la franja horaria } s_j \\ 0 & \text{en caso contrario} \end{cases}$$

i=1,2,…,m representa los autobuses.

j=1,2,…,n representa las franjas horarias disponibles en el taller.

Estas variables son binarias, ya que un autobús solo puede ocupar una franja o no ocupar ninguna.

### *Parámetros*

$d_i$: distancia entre el autobús $a_i$ y el taller.

$p_i$: número de pasajeros que viajan en el autobús $a_i$.

$k_d$: coste por kilómetro recorrido.

$k_p$: penalización por pasajero en caso de no ser atendido.

m: número total de autobuses.

n: número total de franjas horarias disponibles.

### *Función objetivo*

El modelo busca minimizar el coste total derivado del desplazamiento y de la penalización por inasistencia:

$$\text{Min } Z = \sum_{i=1}^{m}\left(k_d d_i\left(\sum_{j=1}^{n} x_{ij}\right) + k_p p_i\left(1 - \sum_{j=1}^{n} x_{ij}\right)\right)$$

### *Restricciones*

1. **Asignación máxima de una franja por autobús**

$$\sum_{j=1}^{n} x_{ij} \leq 1 \; \forall i$$

Cada autobús puede ocupar como máximo una franja horaria.

2. **Capacidad de las franjas horarias**

$$\sum_{i=1}^{m} x_{ij} \leq 1 \; \forall j$$

Cada franja horaria puede ser asignada como máximo a un autobús.

3. **Restricción de dominio**

$$x_{ij} \in \{0,\ 1\} \; \forall i, j$$

Las variables de decisión son binarias.

### *Implementación en GLPK*

Con el objetivo de analizar el impacto económico de las decisiones de mantenimiento y mejorar la planificación de reparaciones, se implementó el modelo de minimización del impacto de averías en GLPK (GNU Linear Programming Kit). Para ello, se organizaron todos los datos del problema en un formato estructurado que facilitara la lectura y comparación de los parámetros. En particular, se definieron los conjuntos de autobuses (I) y franjas horarias (S), lo que permitió representar de forma matricial las posibles combinaciones de asignación entre ambos. Esta estructura simplifica el cálculo del coste total, ya que cada celda $x_{i,s}$ indica si el autobús i es atendido en la franja s (valor 1) o no (valor 0).

La decisión de emplear una variable binaria $x_{i,s}$ en lugar de una variable continua responde a la necesidad de garantizar que cada autobús sólo pueda ocupar una franja de forma exclusiva. Este planteamiento convierte el problema en uno de programación lineal entera binaria (PLEB), más acorde con la naturaleza discreta del sistema de asignación real de la empresa.

Asimismo, se estructuraron los parámetros de entrada —distancias $d_i$ y número de pasajeros $p_i$— como vectores indexados, lo que permite relacionar directamente cada autobús con su coste de desplazamiento y su penalización por no ser atendido. El modelo se apoya en dos constantes globales, $k^d$ y $k^p$, que representan respectivamente el coste por kilómetro recorrido y la penalización económica por pasajero. Esta parametrización flexible facilita ajustar el modelo a distintos escenarios de la compañía, como variaciones en los precios del combustible o en el valor del tiempo de los usuarios.

Inicialmente, se consideró la posibilidad de incluir variables auxiliares o una constante grande M para representar la no asignación de autobuses. Sin embargo, se optó por una formulación más eficiente: la penalización por no asignación se incorporó directamente en la función objetivo mediante el término $k_p p_i(1 - \sum_{s \in S} x_{i,s})$ Con esta estructura se evita el uso de restricciones adicionales, reduciendo el número total de variables y mejorando tanto la claridad como el rendimiento computacional del modelo.

En la práctica, la resolución se realizó utilizando dos ficheros principales: El modelo general (parte-2-1.mod), que contiene la formulación abstracta del problema, incluyendo los conjuntos, parámetros, variables, restricciones y la función objetivo. El script en Python (gen-1.py), encargado de automatizar la ejecución. Este script genera el fichero .dat a partir de un archivo de entrada .in, invoca el solver de GLPK mediante la instrucción glpsol y traduce la salida a un formato legible para el usuario.

Tras la ejecución, el programa muestra en consola tres elementos:

- El valor óptimo de la función objetivo, correspondiente al coste total mínimo.
- El número de variables y restricciones del modelo.
- La asignación final de cada autobús, indicando si fue atendido o no.

Esta salida facilita la validación visual de la coherencia de la solución y permite comprobar que todas las restricciones de capacidad y exclusividad se cumplen correctamente.

Durante el proceso de pruebas, se ejecutaron diferentes escenarios variando el número de autobuses, franjas horarias y los valores de los parámetros kd y kp. En todos los casos, el solver devolvió soluciones lógicas y consistentes con el comportamiento esperado del modelo: los autobuses con menores costes de desplazamiento y mayor número de pasajeros fueron priorizados en la asignación, mientras que aquellos con costes más elevados tendieron a quedar sin atender. Estos resultados confirmaron tanto la validez teórica como la estabilidad práctica de la implementación.

En conjunto, el modelo desarrollado en GLPK demuestra ser válido, robusto y escalable. Su estructura modular permite reutilizarlo con distintos conjuntos de datos, analizar escenarios alternativos de operación y evaluar el impacto económico de diferentes políticas de gestión de averías, sin necesidad de modificar la formulación base. Por tanto, constituye una herramienta eficaz, precisa y transparente para la toma de decisiones operativas en entornos de mantenimiento y transporte.

## Maximización de la satisfacción de los pasajeros

### *Planteamiento del problema*

En esta parte avanzada se propone un modelo cuyo propósito es reducir el impacto sobre los pasajeros evitando, en la medida de lo posible, que dos autobuses con pasajeros en común coinciden en la misma franja horaria. Para ello, se consideran u talleres ($t_1, t_2, \ldots, t_u$), m autobuses ($a_1, a_2, \ldots, a$□) y n franjas horarias disponibles ($s_1, s_2, \ldots, s$□). No todas las franjas están disponibles en todos los talleres.

A diferencia del modelo 2.2.1 (centrado en costes), aquí la satisfacción se modela minimizando los "conflictos" entre autobuses con pasajeros compartidos cuando coinciden en la misma franja (pueden estar en talleres distintos). Cada autobús debe ser asignado exactamente a una franja de algún taller disponible.

### *Variables de decisión*

$$X_{ijs} = \begin{cases} 1 & \text{si el autobús } a_i \text{ se asigna a la franja s del taller } t_j \\ 0 & \text{en caso contrario} \end{cases}$$

$$z_{iks} = \begin{cases} 1 & \text{indica si los autobuses i y k coinciden en la misma franja s} \\ 0 & \text{no está} \end{cases}$$

i = 1,2,…,m representa los autobuses.

k = 1,2,…,m: otro autobús (para las parejas i–k en C)

j = 1,2,...,u representa los talleres.

s = 1,2,…,n representa las franjas horarias disponibles en el taller.

### *Parámetros*

$c_{ik}$: número de pasajeros compartidos o afectados simultáneamente entre los autobuses $a_i$ y ak.

$o_{js}$: parámetro binario que indica si la franja s está disponible (1) o no (0) en el taller t□.

m: número total de autobuses.

u: número total de talleres.

n: número total de franjas horarias.

### *Función objetivo*

La función objetivo de este modelo busca minimizar el impacto total sobre los pasajeros que comparten simultáneamente varios autobuses, es decir, reducir los conflictos que se producen cuando dos autobuses con pasajeros comunes son atendidos en la misma franja horaria, aunque sea en talleres distintos.

$$\boxed{min\ z = \sum_{s=1}^{n} \sum_{i=1}^{m-1} \sum_{k=i+1}^{m} c_{ik}\, z_{iks}}$$

La variable $z_{iks}$ toma el valor 1 sólo cuando los autobuses $i$ y $k$ están asignados a la misma franja s. De esta manera, $z_{iks}$ nos ayuda a contabilizar las situaciones en las que los autobuses coinciden, y, por tanto, deben ser penalizados según el número de pasajeros afectados, representado por $c_{ik}$. Si los autobuses no coinciden en la franja, $z_{iks}$ será igual a 0, y no se añadirá penalización en la función objetivo.

Se introduce la variable auxiliar $z_{iks}$ para linealizar el modelo, ya que el producto entre las variables binarias $x_{ijs}$ $x_{kjs}$ es no lineal y no puede resolverse directamente. La linealización se consigue introduciendo restricciones adicionales que vinculan las variables $x_{ijs}$ y $z_{iks}$, asegurando que la variable $z_{iks}$ solo tomará el valor de 1 cuando ambos autobuses $i$ y $k$ estén asignados a la misma franja s.

### *Restricciones*

Las restricciones garantizan que los autobuses se asignen de forma que se cumplan los límites de capacidad de cada taller y franja, y que no haya más de un autobús asignado a una misma franja en un taller. Las restricciones también incluyen la disponibilidad de franjas horarias. A continuación se detallan las principales restricciones:

1. **Asignación única por autobús:**

$$\boxed{\sum_{j=1}^{u} \sum_{s=1}^{n} x_{ijs} = 1 \quad \forall i}$$ Cada autobús debe estar asignado a una y solo una franja de algún taller.

Esta restricción asegura que cada autobús i esté asignado exactamente a una franja s de algún taller j. No puede haber un autobús que se asigne a varias franjas, y tampoco puede quedar sin ser atendido.

**Capacidad de las franjas:**

$$\boxed{\sum_{i=1}^{m} x_{ijs} \leq 1 \quad \forall j, s}$$ Cada franja de cada taller sólo puede asignarse como máximo a un autobús.

Esta restricción limita la capacidad de cada franja horaria en cada taller. Especifica que cada franja s en el taller $t_j$ puede atender a lo sumo a un único autobús. Esta condición garantiza que no haya solapamientos entre los autobuses en la misma franja y taller.

2. **Disponibilidad de franjas:**

$$x_{ijs} \leq o_{js} \quad \forall ij, s$$

Las franjas no disponibles en un taller no pueden ser utilizadas.

Esta restricción asegura que los autobuses solo sean asignados a franjas disponibles en los talleres. Si una franja no está disponible en un taller determinado, la variable $x_{ijs}$ será igual a 0 para ese autobús i y esa franja s. El parámetro $o_{js}$ toma el valor 1 si la franja s está disponible en el taller $t_j$ y 0 si la franja no está disponible.

3. **Definición de la variable $z_{iks}$**

$$z_{iks} \leq \sum_{j=1}^{u} x_{ijs} \quad \forall i < k, \forall s$$

z no puede ser 1 si i no está en la franja s

$$z_{iks} \leq \sum_{j=1}^{u} x_{kjs} \quad \forall i < k, \forall s$$

z no puede ser 1 si k no está en la franja s

$$z_{iks} \geq \sum_{j=1}^{u} x_{ijs} + \sum_{j=1}^{u} x_{kjs} - 1 \quad \forall i < k, \forall s$$

Si i está en s y k está en s, entonces z debe ser 1

Estas restricciones aseguran que la variable $z_{iks}$ solo sea 1 cuando los autobuses i y k estén asignados en la misma franja s.

El primer conjunto de restricciones garantiza que $z_{iks}$ no será mayor que la asignación de autobuses a la franja s en un taller j.

La última restricción asegura que $z_{iks} = 1$ solo cuando ambos autobuses estén asignados en la misma franja y en el mismo taller, lo que refleja una coincidencia real y evita que autobuses asignados en franjas distintas (o en talleres distintos) se sumen innecesariamente.

4. **Dominio de las variables:**

$$x_{ijs}\ ,\ z_{iks} \in \{0,\ 1\} \quad \forall i, j, k, s$$

Estas restricciones garantizan que cada autobús sea atendido exactamente una vez, que no se solapen las franjas entre distintos talleres y que solo se utilicen aquellas franjas efectivamente disponibles.

## Implementación en GLPK

Para resolver el modelo anterior usamos GLPK/MathProg con un fichero de modelo general parte-2-2.mod y un script gen-2.py que automatiza la creación de datos, la llamada al solver y la presentación del resultado.

Para garantizar el correcto funcionamiento del script en entornos Linux, fue necesario otorgar permisos de ejecución mediante el comando **chmod +x gen-2.py** una vez creado.

El fichero de entrada (.in) sigue el formato especificado en el enunciado, con las dimensiones del problema y las matrices de pasajeros compartidos y disponibilidad de talleres. El script valida tamaños, genera un .dat tabular compatible con MathProg y lanza el solver con:

**glpsol -m parte-2-2.mod -d caso.dat -o solution_2_2.txt**

A partir del fichero de salida, extraemos las asignaciones x = 1 y recalculamos el valor objetivo sumando C[i,k] para cada par de autobuses que comparten franja (la lógica del objetivo). Con esto imprimimos exactamente lo que pide el enunciado:

- Primera línea: <valor_objetivo> <num_variables> <num_restricciones>.
- Resto: una línea por autobús: bus i -> taller j, franja s.

En la formulación práctica, la disponibilidad de franjas se implementa mediante la restricción $x \leq O$, garantizando que no se asignen horarios no disponibles, mientras que la capacidad se controla por pareja (taller,franja), impidiendo que más de un autobús coincida en el mismo espacio temporal. La coincidencia de dos autobuses en la misma franja se modela mediante una variable auxiliar $z_{iks}$ y tres inecuaciones lineales (linealización tipo AND), evitando el uso de productos de variables binarias. Además, para no contabilizar dos veces la misma pareja, solo se consideran las combinaciones $i < k$ en las variables de coincidencia y en la suma del objetivo.

El valor óptimo obtenido representa la mínima coincidencia posible entre autobuses con pasajeros comunes, evidenciando que el modelo logra distribuir las reparaciones de forma eficiente y equitativa. Las franjas asignadas no se solapan, ningún taller supera su capacidad y todas las asignaciones se realizan exclusivamente en horarios disponibles según la matriz O.

En la práctica, esto demuestra que el sistema es capaz de planificar de forma óptima las reparaciones, reduciendo los conflictos operativos y maximizando la satisfacción de los pasajeros, al evitar que varios autobuses con usuarios compartidos estén fuera de servicio simultáneamente. Los resultados confirman la coherencia de la modelización, la corrección de la linealización implementada y la adecuada integración entre las variables de asignación y coincidencia.

En conclusión, la resolución en GLPK valida la eficacia del modelo, demostrando que el enfoque propuesto logra equilibrar la carga de trabajo entre talleres y franjas horarias, reduciendo significativamente el impacto sobre los pasajeros y asegurando la viabilidad de la planificación operativa.

## 2.3. Parte 3: Análisis de Resultados

A lo largo del desarrollo de la práctica se han implementado y probado tres modelos de optimización, cada uno con un nivel creciente de complejidad y precisión. En este apartado se analizan los

resultados obtenidos, se comprueba el cumplimiento de las restricciones establecidas y se evalúa la complejidad y el comportamiento de los modelos ante diferentes escenarios de prueba.

En todos los modelos implementados, las soluciones proporcionadas por GLPK cumplen estrictamente las restricciones del enunciado. En el modelo 2.2.1, cada autobús ocupa como máximo una franja, y cada franja solo se asigna a un autobús, garantizando la exclusividad de atención. En el modelo 2.2.2, las restricciones de capacidad y disponibilidad se respetan en todos los casos: ningún taller supera el límite de un autobús por franja, y las asignaciones se realizan únicamente en las franjas disponibles según la matriz de disponibilidad.

Durante las pruebas, el solver devolvió soluciones factibles y lógicas, confirmando la correcta formulación de las restricciones y la coherencia entre los ficheros de datos generados y el modelo implementado. Además, las variables auxiliares $z_{iks}$ reflejaron de forma precisa las coincidencias de franja, activándose únicamente cuando dos autobuses coincidían en el mismo intervalo horario, tal y como se había diseñado en la linealización.

En el modelo de minimización del impacto de averías (2.2.1), los resultados mostraron que el solver prioriza los autobuses con mayor número de pasajeros y menor coste de desplazamiento. Cuando las franjas disponibles son escasas, el modelo tiende a dejar sin atender a los autobuses más alejados o con menos pasajeros, ya que su penalización es menor en el coste total. Este comportamiento confirma la correcta ponderación entre el coste de desplazamiento y la penalización por inasistencia.

En el modelo de maximización de la satisfacción de los pasajeros (2.2.2), el solver distribuye las asignaciones entre talleres y franjas de manera equilibrada, evitando coincidencias entre autobuses que comparten un número elevado de pasajeros. Al aumentar los valores de la matriz C, el modelo reacciona separando los autobuses conflictivos en franjas distintas, reduciendo el impacto global. En los casos donde las restricciones de disponibilidad o capacidad limitan las combinaciones posibles, el modelo sigue ofreciendo soluciones factibles y mantiene el número de coincidencias en niveles mínimos.

### *Casos de prueba y resultados observados*

Entre los diferentes casos de prueba realizados en el 2.2.2 se incluyeron escenarios con un número reducido de autobuses y talleres, así como otros más amplios con varias franjas horarias y matrices de pasajeros con penalizaciones elevadas entre determinados vehículos.

En los casos más simples, el modelo logró encontrar asignaciones totalmente libres de coincidencias, obteniendo valores óptimos de la función objetivo iguales a cero. En los escenarios intermedios, con mayores interacciones entre autobuses, el solver distribuyó las asignaciones de forma equilibrada, minimizando las coincidencias y obteniendo valores bajos de impacto. Finalmente, en los casos más complejos, donde las restricciones de disponibilidad o capacidad limitaban las combinaciones posibles, el modelo mantuvo su estabilidad y devolvió soluciones factibles, priorizando siempre la reducción del número de pasajeros afectados.

Se ha probado un caso con cinco autobuses, tres franjas horarias y dos talleres, donde uno de los talleres presentaba franjas limitadas y existían penalizaciones elevadas entre algunos autobuses. Al ejecutar el archivo de entrada con el script en Python, GLPK generó los datos y resolvió el modelo de

forma óptima, mostrando en pantalla un valor objetivo de **5**, con **60 variables** y **131 restricciones**.
**python3 gen-2.py caso2.in caso2.dat**

La solución obtenida asignó los autobuses de la siguiente forma:

bus 1 -> taller 1, franja 3
bus 2 -> taller 2, franja 1
bus 3 -> taller 1, franja 2
bus 4 -> taller 2, franja 3
bus 5 -> taller 1, franja 1

El resultado confirma que el modelo distribuye correctamente las asignaciones, minimizando las coincidencias entre autobuses con pasajeros compartidos y cumpliendo todas las restricciones de capacidad y disponibilidad.

En conclusión, los resultados obtenidos validan la consistencia de todos los modelos desarrollados. El paso progresivo de un modelo manual a uno automatizado permitió comprobar cómo la Programación Lineal Entera puede aplicarse de forma eficaz a la optimización de recursos reales.

**Optimización de Problemas en Programación Lineal**

**Realizado por:**
Diego Valladares y María Robledano

**Asignatura:**
Heurística y Optimización

**Profesor:**
Cesar Alberto Escribano Otero

**Institución:**
Universidad Carlos III de Madrid

**Fecha de entrega:**
27/10/2025